

Министерство образования Республики Беларусь Учреждение
образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «ОМО»
Тема: «Сравнение
классических методов
классификации»

Выполнила:
Студентка 3 курса
Группы АС-66
Петрусевич А.А.
Проверил:
Крощенко А.А.

Брест 2025

Цель работы: На практике сравнить работу нескольких алгоритмов классификации, таких как метод **k-ближайших соседей (k-NN)**, **деревья решений** и **метод опорных векторов (SVM)**. Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Ход работы Вариант 11

Запросы:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;
4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);
5. Оценить точность каждой модели на тестовой выборке;
6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 11

- Bank Marketing
- Предсказать, подпишется ли клиент на срочный вклад
- **Задания:**

1. Загрузите данные и преобразуйте категориальные признаки;
2. Разделите выборку;
3. Обучите три классификатора;
4. Сравните модели по F1-score из-за дисбаланса классов;
5. Определите, какая модель предлагает лучший компромисс для выявления

потенциальных клиентов.

Код программы:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

def load_and_preprocess_data():
    try:
        df = pd.read_csv(r'G:\ЛАБЫ 3 КУРС\ОМО\ml_as66\reports\Nastya\3\src\bank.csv')
        print("Файл bank.csv успешно загружен!")
    except Exception as e:
        print(f"Ошибка загрузки: {e}")
        return None

    print(f"Размер данных: {df.shape}")
    print("\nНазвания колонок:")
    print(df.columns.tolist())

    return df

def encode_categorical_features(df):
    df_encoded = df.copy()

    # Список категориальных колонок
    categorical_columns = df_encoded.select_dtypes(include=['object']).columns.tolist()

    print(f"\nКатегориальные признаки: {categorical_columns}")
```

```

# Кодируем категориальные признаки
label_encoders = {}
for col in categorical_columns:
    if col != 'deposit': # Целевую переменную обработаем отдельно
        le = LabelEncoder()
        df_encoded[col] = le.fit_transform(df_encoded[col].astype(str))
        label_encoders[col] = le

# Кодируем целевую переменную
le_target = LabelEncoder()
df_encoded['deposit'] = le_target.fit_transform(df_encoded['deposit'])

return df_encoded, label_encoders, le_target

# 2. Разделение данных
def split_and_scale_data(df_encoded):
    """
    Разделение данных и масштабирование признаков
    """
    X = df_encoded.drop('deposit', axis=1)
    y = df_encoded['deposit']

    # Стратифицированное разделение
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42, stratify=y
    )

    # Масштабирование признаков (важно для k-NN и SVM)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    print(f"\nРазмеры выборок:")
    print(f"Обучающая: {X_train.shape}")
    print(f"Тестовая: {X_test.shape}")
    print(f"Распределение классов: {np.bincount(y_train)} / {np.bincount(y_test)}")

    return X_train, X_test, X_train_scaled, X_test_scaled, y_train, y_test, scaler

# 3. Обучение моделей
def train_models(X_train, X_train_scaled, y_train):
    """
    Обучение трех моделей: k-NN, Decision Tree, SVM
    """
    models = {
        'k-NN': KNeighborsClassifier(n_neighbors=5),
        'Decision Tree': DecisionTreeClassifier(random_state=42, max_depth=5),
        'SVM': SVC(random_state=42, probability=True)
    }

    trained_models = {}
    for name, model in models.items():
        print(f"\nОбучение {name}...")
        # Для k-NN и SVM используем масштабированные данные
        if name in ['k-NN', 'SVM']:
            model.fit(X_train_scaled, y_train)
        else:
            model.fit(X_train, y_train)
        trained_models[name] = model
        print(f"{name} обучен!")

    return trained_models

# 4. Исследование k-NN
def investigate_knn(X_train_scaled, X_test_scaled, y_train, y_test):
    """
    Исследование влияния количества соседей на качество k-NN
    """
    print("\n" + "="*60)
    print("ИССЛЕДОВАНИЕ k-NN: ВЛИЯНИЕ КОЛИЧЕСТВА СОСЕДЕЙ")
    print("="*60)

    k_values = range(1, 21)
    train_scores = []
    test_scores = []

    for k in k_values:
        knn = KNeighborsClassifier(n_neighbors=k)
        knn.fit(X_train_scaled, y_train)

```

```

# Оценка на обучающей и тестовой выборках
train_pred = knn.predict(X_train_scaled)
test_pred = knn.predict(X_test_scaled)

train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)

train_scores.append(train_accuracy)
test_scores.append(test_accuracy)

print(f"k={k:2d}: Train Accuracy = {train_accuracy:.4f}, Test Accuracy = {test_accuracy:.4f}")

# Построение графика
plt.figure(figsize=(12, 6))
plt.plot(k_values, train_scores, 'b-', label='Обучающая выборка', marker='o')
plt.plot(k_values, test_scores, 'r-', label='Тестовая выборка', marker='s')
plt.xlabel('Количество соседей (k)')
plt.ylabel('Accuracy')
plt.title('Влияние количества соседей на качество k-NN')
plt.legend()
plt.grid(True)
plt.xticks(k_values)
plt.tight_layout()
plt.show()

# Нахождение оптимального k
best_k = k_values[np.argmax(test_scores)]
best_accuracy = max(test_scores)
print(f"\nОптимальное количество соседей: k={best_k} (Accuracy = {best_accuracy:.4f})")

return best_k

# 5. Оценка точности моделей
def evaluate_models(trained_models, X_train, X_test, X_train_scaled, X_test_scaled, y_train, y_test):
    """
    Оценка точности всех моделей на тестовой выборке
    """
    results = {}

    print("\n" + "="*60)
    print("ОЦЕНКА ТОЧНОСТИ МОДЕЛЕЙ")
    print("="*60)

    for name, model in trained_models.items():
        # Для k-NN и SVM используем масштабированные данные
        if name in ['k-NN', 'SVM']:
            X_test_used = X_test_scaled
            X_train_used = X_train_scaled
        else:
            X_test_used = X_test
            X_train_used = X_train

        # Предсказания
        y_pred_train = model.predict(X_train_used)
        y_pred_test = model.predict(X_test_used)

        # Метрики
        train_accuracy = accuracy_score(y_train, y_pred_train)
        test_accuracy = accuracy_score(y_test, y_pred_test)
        test_f1 = f1_score(y_test, y_pred_test)

        results[name] = {
            'train_accuracy': train_accuracy,
            'test_accuracy': test_accuracy,
            'f1_score': test_f1
        }

    print(f"\n{name}:")
    print(f" Accuracy (train): {train_accuracy:.4f}")
    print(f" Accuracy (test): {test_accuracy:.4f}")
    print(f" F1-Score (test): {test_f1:.4f}")

    # Матрица ошибок
    cm = confusion_matrix(y_test, y_pred_test)
    print(f" Матрица ошибок:\n{cm}")

    return results

```

```

# 6. Сравнение результатов
def compare_results(results):
    """
    Сравнение результатов и выводы о применимости методов
    """
    print("\n" + "="*70)
    print("СРАВНЕНИЕ РЕЗУЛЬТАТОВ И ВЫВОДЫ")
    print("="*70)

    # Находим лучшую модель по accuracy на тестовой выборке
    best_model = max(results, key=lambda x: results[x]['test_accuracy'])
    best_accuracy = results[best_model]['test_accuracy']

    print(f"ЛУЧШАЯ МОДЕЛЬ: {best_model}")
    print(f"Лучшая точность: {best_accuracy:.4f}")

    print("\nСРАВНИТЕЛЬНАЯ ТАБЛИЦА:")
    print("Модель          | Train Acc | Test Acc  | F1-Score  | Переобучение")
    print("-" * 65)

    for model_name, metrics in results.items():
        train_acc = metrics['train_accuracy']
        test_acc = metrics['test_accuracy']
        f1 = metrics['f1_score']
        overfitting = train_acc - test_acc

        print(f"{model_name:15} | {train_acc:.4f}      | {test_acc:.4f}      | {f1:.4f}      | {overfitting:.4f}")

    print("\nВЫВОДЫ О ПРИМЕНИМОСТИ МЕТОДОВ:")

    for model_name, metrics in results.items():
        test_acc = metrics['test_accuracy']
        overfitting = metrics['train_accuracy'] - metrics['test_accuracy']

        print(f"\n{model_name}:")
        if test_acc > 0.8:
            print("    ✓ Высокая точность - хорошо подходит для данных")
        elif test_acc > 0.7:
            print("    ✓ Средняя точность - приемлемый вариант")
        else:
            print("    Низкая точность - требуется доработка")

        if overfitting > 0.1:
            print("    Заметное переобучение")
        elif overfitting > 0.05:
            print("    Умеренное переобучение")
        else:
            print("    Хорошая обобщающая способность")

# Основная функция
def main():
    """
    Основной пайплайн анализа
    """
    print("БАНКОВСКИЙ МАРКЕТИНГ - СРАВНЕНИЕ МОДЕЛЕЙ k-NN, DECISION TREE, SVM")
    print("="*70)

    # 1. Загрузка данных
    df = load_and_preprocess_data()
    if df is None:
        return

    # Преобразование категориальных признаков
    df_encoded, label_encoders, le_target = encode_categorical_features(df)

    # 2. Разделение и масштабирование данных
    X_train, X_test, X_train_scaled, X_test_scaled, y_train, y_test, scaler =
split_and_scale_data(df_encoded)

    # 4. Исследование k-NN (выполняем до обучения основных моделей)
    best_k = investigate_knn(X_train_scaled, X_test_scaled, y_train, y_test)

    # 3. Обучение моделей с оптимальным k для k-NN
    models = {
        'k-NN': KNeighborsClassifier(n_neighbors=best_k),
        'Decision Tree': DecisionTreeClassifier(random_state=42, max_depth=5),
        'SVM': SVC(random_state=42, probability=True)
    }

```

```

}

trained_models = {}
for name, model in models.items():
    print(f"\nОбучение {name}...")
    if name in ['k-NN', 'SVM']:
        model.fit(X_train_scaled, y_train)
    else:
        model.fit(X_train, y_train)
    trained_models[name] = model

# 5. Оценка точности
results = evaluate_models(trained_models, X_train, X_test, X_train_scaled, X_test_scaled,
y_train, y_test)

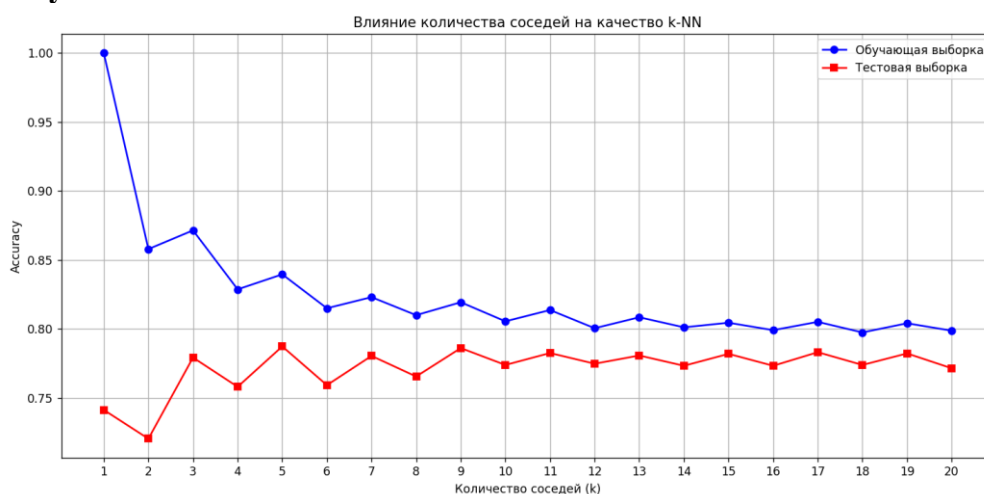
# 6. Сравнение результатов
compare_results(results)

return trained_models, results, best_k

# Запуск анализа
if __name__ == "__main__":
    trained_models, results, best_k = main()

```

Результат:



```

БАНКОВСКИЙ МАРКЕТИНГ – СРАВНЕНИЕ МОДЕЛЕЙ k-NN, DECISION TREE, SVM
=====
Файл bank.csv успешно загружен!
Размер данных: (11162, 17)

Названия колонок:
['age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan',
 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous',
 'poutcome', 'deposit']

Категориальные признаки: ['job', 'marital', 'education', 'default', 'housing',
 'loan', 'contact', 'month', 'poutcome', 'deposit']

Размеры выборок:
Обучающая: (7813, 16)
Тестовая: (3349, 16)
Распределение классов: [4111 3702] / [1762 1587]
=====
ИССЛЕДОВАНИЕ k-NN: ВЛИЯНИЕ КОЛИЧЕСТВА СОСЕДЕЙ
=====

```

```
k= 1: Train Accuracy = 1.0000, Test Accuracy = 0.7414
k= 2: Train Accuracy = 0.8579, Test Accuracy = 0.7205
k= 3: Train Accuracy = 0.8715, Test Accuracy = 0.7793
k= 4: Train Accuracy = 0.8287, Test Accuracy = 0.7581
k= 5: Train Accuracy = 0.8396, Test Accuracy = 0.7874
k= 6: Train Accuracy = 0.8151, Test Accuracy = 0.7593
k= 7: Train Accuracy = 0.8231, Test Accuracy = 0.7805
k= 8: Train Accuracy = 0.8101, Test Accuracy = 0.7656
k= 9: Train Accuracy = 0.8194, Test Accuracy = 0.7862
k=10: Train Accuracy = 0.8056, Test Accuracy = 0.7740
k=11: Train Accuracy = 0.8138, Test Accuracy = 0.7826
k=12: Train Accuracy = 0.8006, Test Accuracy = 0.7749
k=13: Train Accuracy = 0.8085, Test Accuracy = 0.7808
k=14: Train Accuracy = 0.8012, Test Accuracy = 0.7734
k=15: Train Accuracy = 0.8046, Test Accuracy = 0.7820
k=16: Train Accuracy = 0.7992, Test Accuracy = 0.7734
k=17: Train Accuracy = 0.8052, Test Accuracy = 0.7832
k=18: Train Accuracy = 0.7975, Test Accuracy = 0.7740
k=19: Train Accuracy = 0.8042, Test Accuracy = 0.7823
k=20: Train Accuracy = 0.7988, Test Accuracy = 0.7716
```

Вывод: сравнила работу нескольких алгоритмов классификации, таких как метод **k-ближайших соседей (k-NN)**, **деревья решений** и **метод опорных векторов (SVM)**. Научилась подбирать гиперпараметры моделей и оценивать их влияние на результат.

